CASE STUDY · AI AUTOMATION & TECHNICAL DOCUMENTATION

AI Recruiting Automation Platform

A DeepSeek matching engine and a 14-service communications pipeline, designed and built from scratch, running unattended in production.

AI Automations Expert | Senior Technical Writer | LLM Workflows & APIs

Wessam Mandour

THE PROBLEM

Recruiting top-of-funnel doesn't scale by hand

Hundreds of candidates per role. Multi-channel outreach under provider rate caps. Interview transcripts to score. And a final match a recruiter must actually trust.

Volume

Hundreds of applicants per role, most not a fit, manual screening is hours of work.

Rate limits

Email + WhatsApp outreach must respect provider caps and never double-send.

Transcripts

Screening interviews arrive as raw transcripts that need consistent scoring.

Trust

A bare match score is useless, a recruiter needs to know WHY a candidate ranked.

THE SYSTEM · TWO INDEPENDENT PROJECTS

Two separate projects, one funnel

◆ Role Matching

`Project 01 · Python · FastAPI`

Independent project. Ranks candidates through a six-stage funnel, scoring upward of 20,000 per run (20,000+ LLM API calls). 90 to 120 min on Railway, with Slack status to the team.

◆ Candidate Flow

`Project 02 · Node · Express · 14 svc`

Independent project. Webhook automations plus cron reminder sweeps. 2,000+ WhatsApp and email a day; CV parsed in under a minute; a completed interview updates Notion in about 60 seconds.

40,000
rows scanned / run

20,000+
scored / run (LLM API)

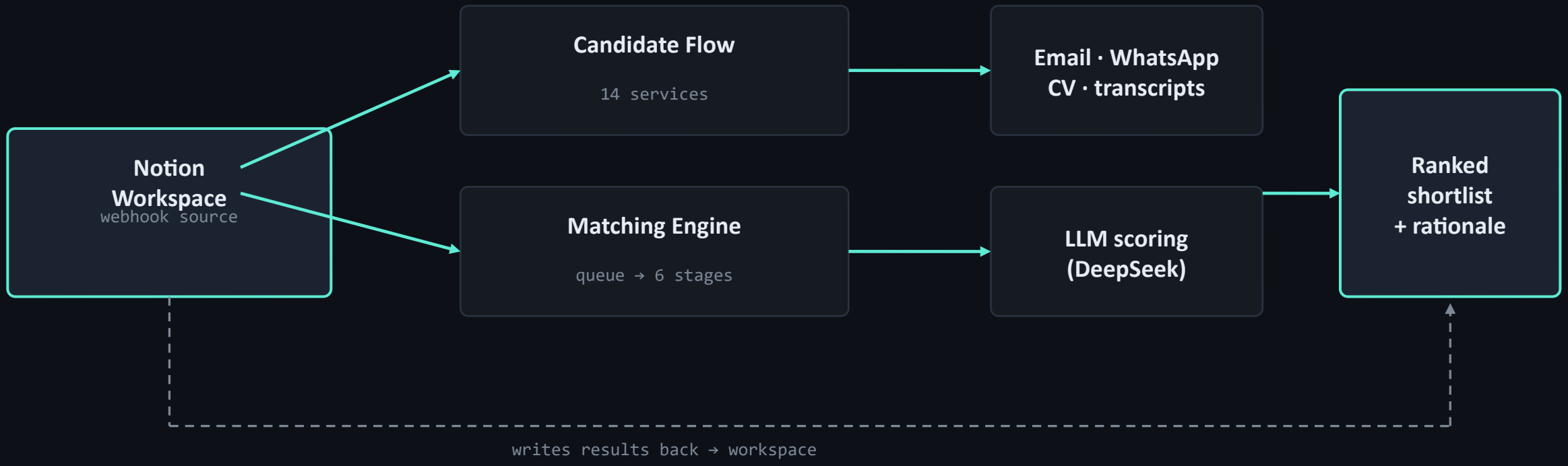
90-120 min
matching run on Railway

2,000+/day
WhatsApp + email

Deployed on Railway, triggered by webhooks and cron schedules, with Slack status to the whole team.

ARCHITECTURE

Event-driven, with the workspace as source of truth



No shared database to corrupt · failure in one service can't block another · every service stateless & idempotent

DEEP DIVE · MATCHING ENGINE

Six stages: job description → ranked shortlist

Each stage narrows the pool before the next, more expensive stage runs. The LLM only sees candidates worth paying to score.

1

Rubric extraction

LLM turns free-text job spec into a structured rubric: gates, requirements, keywords.

2

Keyword reduction

Cheap local filter drops no-signal candidates BEFORE any paid LLM call, top cost lever.

3

Hard filters

Non-negotiable gates: language, location, minimum screening score.

4

Priority ordering

Bucket by business priority; score the highest-value candidates first.

5

LLM scoring

0-100 + written rationale each, concurrency-capped, retried, failure-isolated.

6

Qualify & backfill

Link qualified first; backfill best below-threshold to hit target, logged.

UNDER THE HOOD

The parts that make it production, not a script

Idempotent re-runs

Scored records skipped on re-run, no double-contact, no duplicate matches.

Rate limiting

Sequential queue, concurrency cap, write delays, under quota by design.

Graceful degradation

Backoff + JSON hardening; one bad candidate can't sink a 200-candidate run.

Schema-drift defense

Missing props ignored, odd types logged & skipped, eligibility re-checked.

AI only where needed

DeepSeek for judgment; regex for parsing, filtering, routing. No wasted tokens.

Observability + Slack

Logs keyed by id, email, stage; Slack status to the team at start and finish.

RELIABILITY & ERROR HANDLING

What survives when a third party misbehaves

Project 01 · Role Matching

- DeepSeek over HTTPX: 7 retries, Retry-After aware, 5xx retried, other 4xx fail-fast, 90s timeout
- Notion: 10 attempts in a 180s window, longer backoff on 503, invalid cursor = 5-min cooldown + restart
- Global async semaphore (100) + prefetch window; sequential job queue with a 5-min cooldown between jobs
- Per-candidate isolation; the run fails only if zero usable scores come back; idempotent re-runs

Project 02 · Candidate Flow

- Postmark: hourly + daily send caps (sliding window), 408/429/5xx backoff, 5-min queue cooldown
- Hireflix transcript: polls up to 10 min at 60s, writes only when every answer is transcribed
- Twilio: cron-driven, five templates, a configurable hours-since threshold, never double-nudges
- CV: pdf-parse + regex extraction, DeepSeek only to structure & score; SIGTERM + crash handlers

Drawn from the actual codebases, not aspirational.

CONFIGURABLE BY DESIGN

One codebase, many deployments, zero forks

Nothing is hard-coded per client. Behavior is driven entirely by environment variables, so the same service is deployed many times, each tuned to a different goal, by changing config and redeploying.

Same engine, Flash or Pro

One codebase, two deployments: DeepSeek Flash for speed and cost, DeepSeek Pro for tougher roles. A single env var picks the model.

One reminder, deployed 3×

The WhatsApp reminder service runs three times, each with a different "hours since submission" threshold, for a staggered reminder sequence.

Same code, new database

Each branch points at a different Notion database in the same workspace via env vars. New database, new deploy, no code change.

Same automation, different database, setting, or timeframe? Redeployed to a new space in about 5 minutes.

TECHNICAL WRITING

The documentation is part of the deliverable

Every service ships an operational handbook. For a system that runs unattended, the docs are the reliability layer.

Audience-layered

One-table overview for managers, branch-level depth for developers, same doc.

Operational

Documents which log line proves which behavior: debug by grep, not by reading source.

Failure-first

Maps real symptoms to the exact gate that caused them.

troubleshooting.md

```
## Troubleshooting

Candidate not considered for a role, check, in order:
  1. Stage-2 keyword overlap
  2. Stage-3 hard filters (language, location, score)
  3. existing scored row (already processed)
  4. duplicate-email de-duplication

Fewer qualified matches than target, expected:
  Stage 6 backfills below-threshold by descending
  score and logs
fallback_selected=true per row.
```

IMPACT

What the automation delivers

40,000

candidate rows scanned per matching run

20,000+

scored per run, 20k+ LLM API calls

90-120 min

full run on Railway, with Slack status

2,000+/day

WhatsApp + emails (Twilio + Postmark)

Manual work that took weeks, now seconds to a couple of hours. Workflow on the previous slides.

WHAT THIS DEMONSTRATES

AI automation

14 event-driven services, idempotent, rate-limited, running unattended in production.

Technical writing

Audience-layered operational handbooks, failure-first troubleshooting, this case study.

Cost-aware AI

DeepSeek on judgment-heavy steps, regex everywhere else. No wasted tokens.

Wessam Mandour

wessam.mandour94@gmail.com · linkedin.com/in/wessam-mandour · github.com/Wes-200